

docker_class_3(46)

launched ec2 cand connected

install docker

yum install docker -y && systemctl start docker

Download the official NGINX image from Docker Hub:

docker pull nginx

Verify Downloaded Images List all available Docker images:

docker images

Start a container from the NGINX image : to start the container after create will not enter container

****docker run -it -d --name cont-1 nginx**

- **-d** → Runs the container in **detached mode** (background)
- **-it** → Allocates interactive terminal (not really needed when using **-d**)

-name cont-1 → Assigns a name to the container

when container create background source code will be there

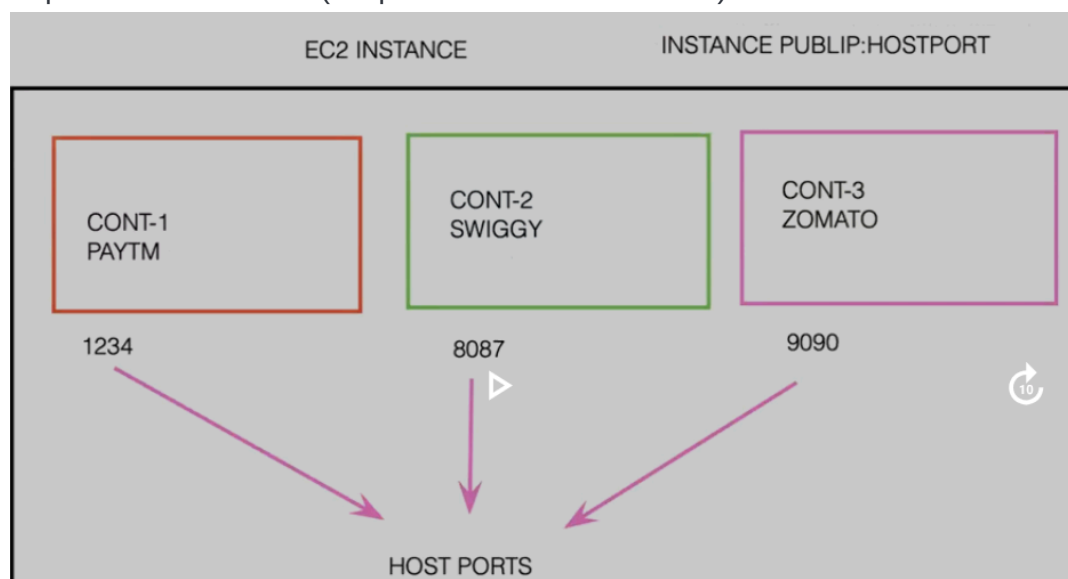
Inspect Container Details: To get detailed information (including IP address):

docker inspect cont-1

to access the application you can get ip `docker inspect container_name` and give curl ip address and curl meaning if the url is working or not This works **only inside the host machine**

generally if we give ip address it wont work we have to give ports to container then only it will work please check below

Expose Container Port (Required for Browser Access)



Run container with port mapping:

docker run -it -d --name cont-2 -p 9090:80 nginx

```
-p 9090:80
```

9090 : hostport —> used to access the application which is running inside the container

80 : containerPort —> it depends on the image

public ip address with port 9090 you can access this

Now open: `http://<server-ip>:9090`

one more create container :

```
docker run -itd --name cont-4 -p 2222:80 shaikmustafa/mygame
```

2222 → Host machine port

80 → Container port (NGINX default)

Now open: `http://<server-ip>:2222`

Port mapping = Door to access container

Docker Container Access & Custom Image Guide

I want to enter inside container previously i used

Attaching to a Running Container

You can attach to a running container using:

```
docker attach cont-2
```

it will show below one

```
harish@aold2402006:~$ docker attach cont-4
2026/03/17 19:24:22 [notice] 34#34: signal 28 (SIGWINCH) received
2026/03/17 19:24:22 [notice] 32#32: signal 28 (SIGWINCH) received
2026/03/17 19:24:22 [notice] 37#37: signal 28 (SIGWINCH) received
```

Behavior:

- Connects to the container's **main process (STDIN/STDOUT)**
- You will see the live output of the container
- If you press `Ctrl + C`, it may **stop the container**

⚠ Problem:

- This is **dangerous for production**
- You can accidentally stop the container

If the container is stopped:

```
docker start cont-2
```

Running Commands Without Entering Container

docker exec: is without enter container we can perform the commands

ex: docker exec cont-2 ls

we can see the list of files

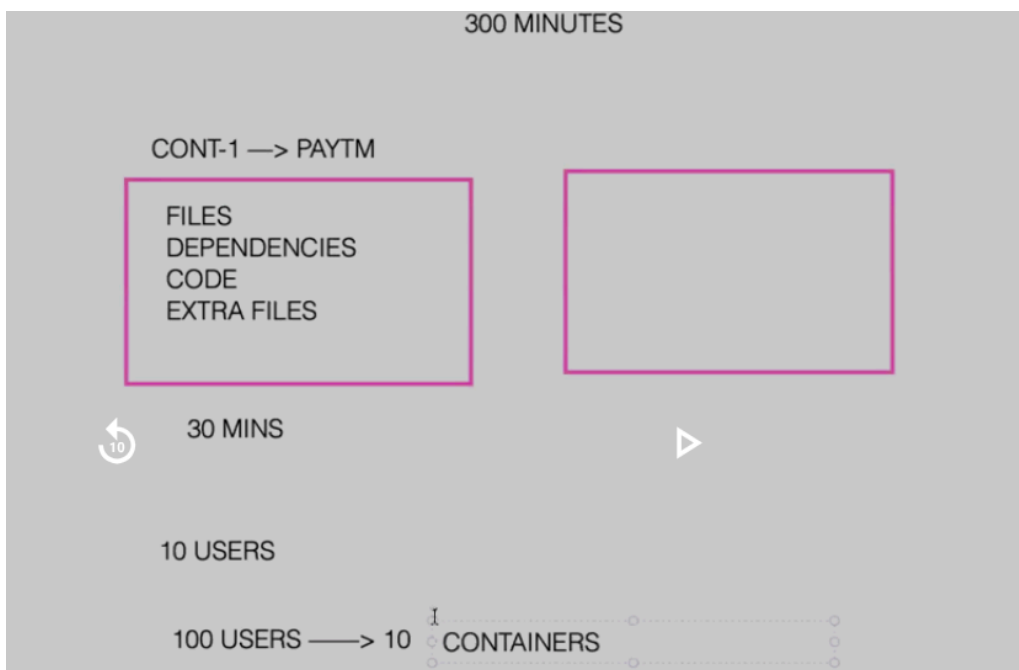
docker exec cont-2 mkdir FLM : to create a folder

docker exec cont-2 : to check the status

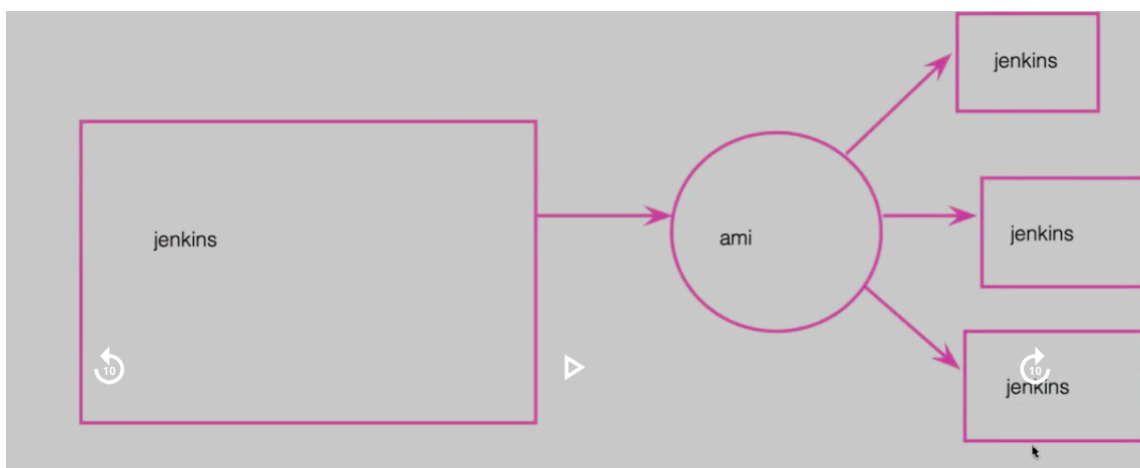
Entering Inside Container

To open a shell inside the container:

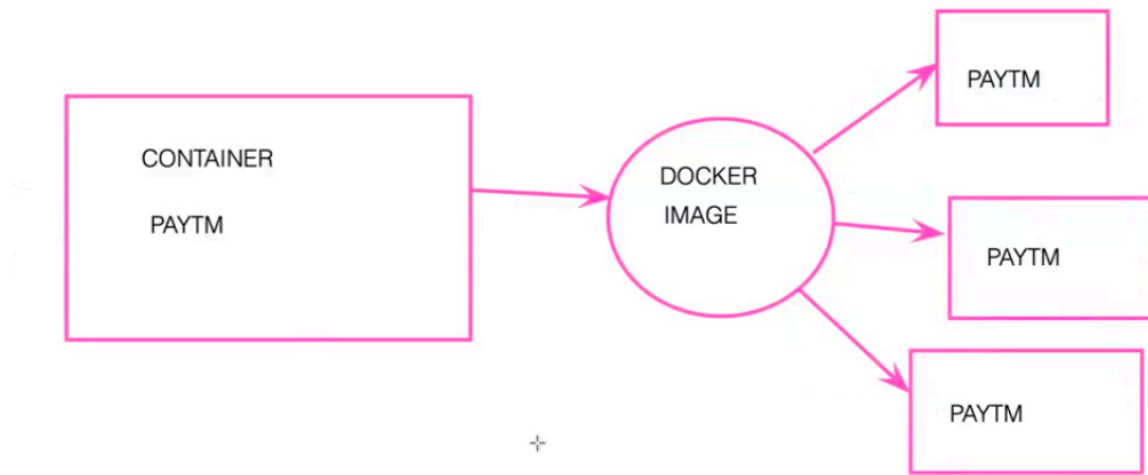
docker exec -it cont-2 /bin/bash : to enter the container



to create 1 cont 30 min this container using 10 people if 100 people using this container it required 9 more containers i.e $10 \times 30 = 300\text{min}$



same i will follow this above concept here



1. first i will run application i.e container using this i will create own image using this image i will create own container here also paytm will come here it will create own container

Step 1: Run a Base Container

ex: using ubuntu i want to create container

`docker run -it --name paytm ubuntu --> enter -->iam inside container paym`

Step 2: Create Files Inside Container

creating 5 files here

`touch paytmfiles{1..5}`

`ls`

created paytm files

Step 3: Exit the Container

`exit`

`docker ps` : in this if we have paytm through using paytm container name

Create Image from Container

`docker commit paytm new-image-name` : to create image using container

through created image will create few container/Create New Containers and we will get all files

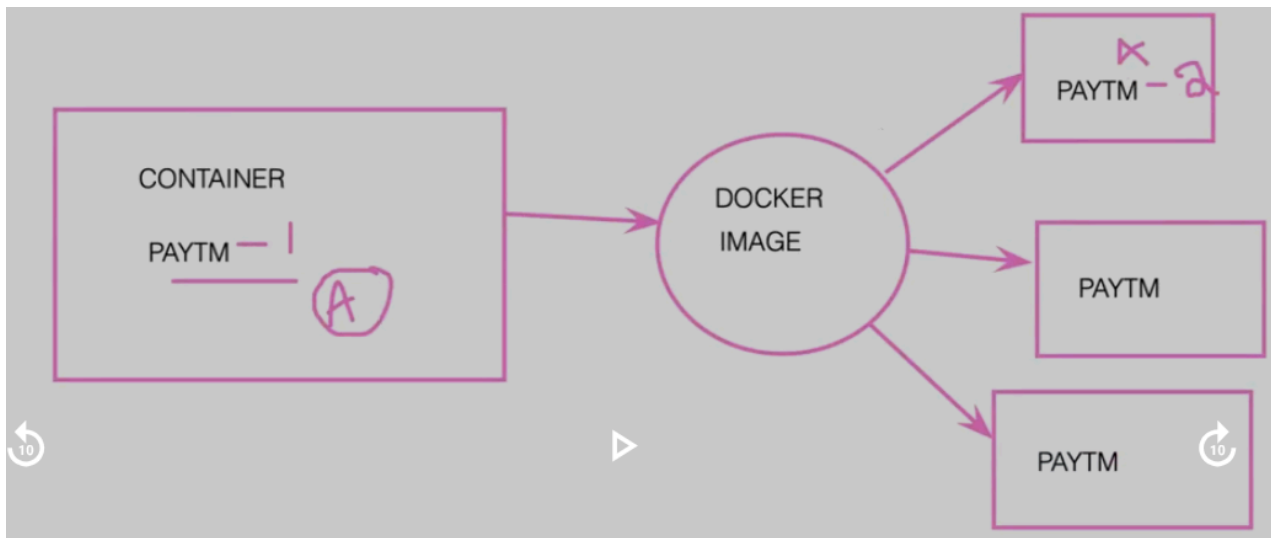
`docker run -it --name paytm-1 new-image-name`

Check files: `ls`

You should see: `paytmfiles1 paytmfiles2 paytmfiles3 paytmfiles4 paytmfiles5`

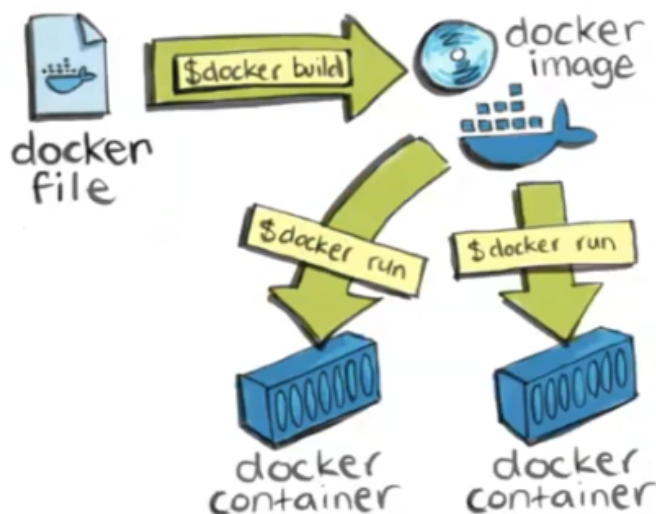
****Create More Containers**

docker run -it --name paytm-4 new-image-name
 docker run -it --name paytm-5 new-image-name



My query is if from container one i.e paytm-1 if i add any data or files will it come container 2 and container 3 Answer is : No data will not replicate through using volume possible

Dockerfile: It is basically a text file which contains some set of instructions why we are using this means: to create image we use Dockerfile to automate Docker Image Always D is the capital for Dockerfile How it works: first you need to create a Docker



o

file here set of instruction will pass now if i build Dockerfile image will create if image run container will run

Docker File Components:

FROM: this component is used to define images in docker file ex: FROM ubuntu , FROM nginx.

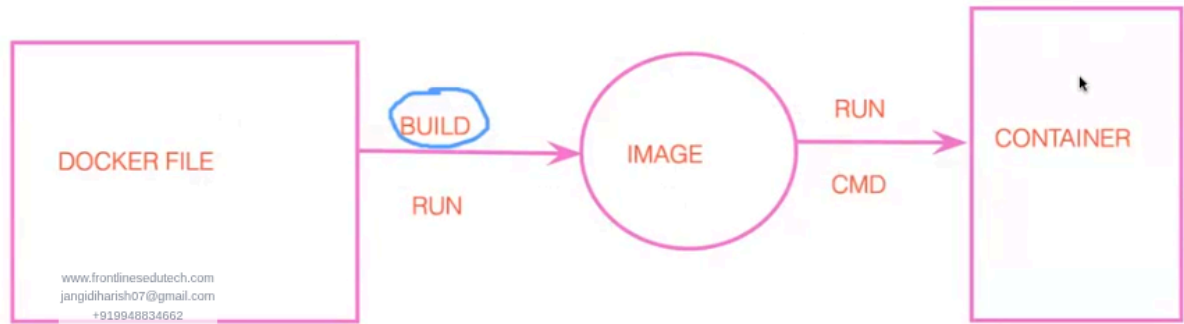
LABEL: This is used to define author of the docker file ex: name:mustafa

COPY: used to copy the files from local system to container and copy will copy only files and folder ex: COPY source_file destination_file

ADD: used to copy the files from our local to container and here it will download the files from

internet and send to container
command inside

ex: ADD url destination_container RUN: it is used to perform a



o

container while we build the image
used t

ex: RUN touch aws ,RUN yum install git -y CMD: it is also

Dockerfile – How It Works

A Dockerfile is a set of instructions used to build a Docker image.

- o First, you **write a Dockerfile**
- o Then you **build the image**
- o Finally, you **run a container from that image**

Dockerfile → Image → Container

Dockerfile Components (Correct & Clean)

FROM

Defines the base image.

Example:

```
FROM ubuntu
FROM nginx
```

LABEL

Adds metadata (like author info).

Example:

```
LABEL maintainer="mustafa"
```

COPY

Copies files from local system to the container.

- Only copies local files/folders

Example:

```
COPY app.js /app/
```

ADD

Similar to COPY but with extra features:

- Can download files from a URL
- Can extract compressed files

Example:

```
ADD https://example.com/file.zip /app/
```

👉 In most cases, prefer `COPY` over `ADD`

RUN

Executes commands while **building the image**

Example:

```
RUN apt-get update && apt-get install -y git
```

CMD

Defines the default command when a container starts.

Example:

```
CMD ["node", "app.js"]
```

ENTRYPOINT

Similar to CMD, but has **higher priority**

- Cannot be easily overridden
- Used when container must always run a specific command

Example:

```
ENTRYPOINT ["node"]
```

WORKDIR

Sets the working directory inside the container.

Example:

```
WORKDIR /app
```

ENV

Defines environment variables inside the container.

- Available at runtime

Example:

```
ENV NODE_ENV=production
```

ARG

Used for build-time variables.

- Only available during image build
- Not available at runtime

Example:

```
ARG VERSION=1.0
```

EXPOSE

Documents which port the container uses.

 Important:

- Does NOT actually open the port
- Used for reference only

Example:

```
EXPOSE 3000
```

Key Differences

RUN vs CMD

- RUN → executes during build
- CMD → executes when container starts

CMD vs ENTRYPOINT

- CMD → can be overridden

- ENTRYPOINT → fixed command (higher priority)

ENV vs ARG

- ENV → available inside container at runtime
 - ARG → only available during build
-

Summary

- Dockerfile defines how to build an image
- Image is used to run containers
- Instructions like RUN, CMD, and ENTRYPOINT control behavior at different stages

Build Stage → RUN, ARG

Runtime Stage → CMD, ENTRYPOINT, ENV
